1 Recap: Perfect Secrecy

Recall from last time:

- Shannon secrecy says that the *a posteriori* distribution over the message, given the ciphertext, is identical to the *a priori* distribution. (See the lecture notes for formal definition.) Informally, "seeing the ciphertext is only as good as seeing nothing at all."
- Perfect secrecy is the property that the distribution of the ciphertext $c \in \mathcal{C}$ (over the choice of key $k \in \mathcal{K}$) is exactly the same, no matter what message $m \in \mathcal{M}$ was encrypted.
- Shannon secrecy and perfect secrecy are equivalent.
- The one-time pad scheme enjoys Shannon/perfect secrecy.

2 Limits of Shannon/Perfect Secrecy

For reference, here again is the definition of Shannon secrecy:

Definition 2.1 (Shannon secrecy). A shared-key encryption scheme $(\mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec})$ with message space $\mathcal{M}$ and ciphertext space $\mathcal{C}$ is *Shannon secret with respect to a probability distribution D* over $\mathcal{M}$ if for all $\bar{m} \in \mathcal{M}$ and all $\bar{c} \in \mathcal{C}$,

$$\Pr_{m \leftarrow D,\, k \leftarrow \mathsf{Gen}}[m = \bar{m} \mid \mathsf{Enc}_k(m) = \bar{c}] = \Pr_{m \leftarrow D}[m = \bar{m}]. \tag{2.1}$$

The scheme is *Shannon secret* if it is Shannon secret with respect to every distribution D over $\mathcal{M}$.

Now that we know that the one-time pad is Shannon secret, what more is left to do? (Cancel the course?) Well, first notice that the one-time pad scheme is not necessarily very usable: the key length is as large as the message length, and for long messages it may be unrealistic for Alice and Bob to establish (and keep secret!) a huge key before communicating. (This is not to mention the additional distinct keys needed for when Carol, Dan, Edith, and Fred enter the picture. . .) Also notice that the key can by used only *one time*: encrypting more than one message with the same key is "abusing the model," and causes the security proof to break down. In fact, using the key more than once often makes the scheme *completely insecure*. (The USSR learned this the hard way as a result of the US/UK-led "Venona" project during the Cold War.)

It turns out that these drawbacks are not really a deficiency of the one-time pad itself, but are *inherent* in the strong requirement of perfect secrecy.

Theorem 2.2 (Shannon's theorem). *If a shared-key encryption scheme (with key space $\mathcal{K}$ and message space $\mathcal{M}$) is Shannon secret, then $|\mathcal{K}| \geq |\mathcal{M}|$.*

Proof. We prove the contrapositive: supposing that $|\mathcal{K}| < |\mathcal{M}|$ for some scheme $(\mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec})$, we will show that the scheme is not Shannon secret. It suffices to show the existence of a distribution D over $\mathcal{M}$, a fixed message $\bar{m} \in \mathcal{M}$, and a ciphertext $\bar{c} \in \mathcal{C}$ such that the two sides of Equation (2.1) are unequal.

Let D be the uniform distribution over $\mathcal{M}$, let $m \in \mathcal{M}$ be arbitrary, let $\bar{k} \in \mathcal{K}$ be an arbitrary key in the support of Gen (i.e., Gen outputs $\bar{k}$ with probability > 0), and let $\bar{c}$ be any ciphertext in the support of $\mathsf{Enc}_{\bar{k}}(m)$ (keeping in the mind that Enc may be randomized). Now assume without loss of generality that Dec is deterministic (exercise: prove that it's really w.l.o.g., using the correctness of decryption), and define

$\mathcal{D} = \{\mathsf{Dec}_k(\bar{c}) \ : \ k \in \mathcal{K}\} \subseteq \mathcal{M}$ to be the set of all possible decryptions of $\bar{c}$ under all possible keys. We must have $|\mathcal{D}| \leq |\mathcal{K}| < |\mathcal{M}|$, which implies that there exists some message $\bar{m} \in \mathcal{M} \setminus \mathcal{D}$. By correctness of decryption, and because D is the uniform distribution over $\mathcal{M}$, we have

$$0 = \Pr_{m \leftarrow D, k \leftarrow \mathsf{Gen}}[m = \bar{m} \mid \mathsf{Enc}_k(m) = \bar{c}] < \Pr_{m \leftarrow D}[m = \bar{m}] = \frac{1}{|\mathcal{M}|},$$

as desired. □

Notice that the proof above is "effective" in the sense that $\bar{m}$ can be computed by explicitly enumerating $\mathsf{Dec}_k(\bar{c})$ for every $k \in \mathcal{K}$. However, if $\mathcal{K}$ is huge, this task may take too long to be feasible.

3 Roadmap to Computational Security

We have seen that perfect secrecy is obtainable, but impractical — the key must be as long as the message. Moreover, the proof of this fact provides an actual attack, which enumerates all possible keys. On the one hand, if the key space is huge (say, 2^{1000}), the attack may need to run for a long, long time. On the other hand, maybe some schemes fall to faster attacks that don't have to enumerate all keys...

Today (and throughout most of the rest of course), we will be focused on schemes that are meaningfully secure (though not "perfectly" so) *as long as the adversary does not employ an absurdly large amount of computation*. We have good reason to believe that there are inherent limits on the amount of computation that can be performed in the physical universe — for example, the number of atoms in the observable universe is estimated at around 10^{80}, and we have only about 10^{10} years until our sun runs out of fuel and collapses.

The *computationally secure* schemes we develop will have keys much shorter than the message length, but this is really just the beginning: they will also have absurdly strong (seemingly even paradoxical) security and functionality properties that go far beyond just keeping a message secret.

To get there from here, there are many questions to which we must give precise mathematical answers. here are just a few:

- How do we *model* "bounded computation"?
- What does it mean for a problem to be *hard* for bounded computation?
- Where might we hope to *find* such hard problems?
- How do we *define* security against computationally bounded adversaries?
- How do we *use* hard problems to design schemes that satisfy our security definitions?

To keep a concrete example in mind, perhaps you have heard the (informal) conjecture that "factoring is hard." We will see how to state this conjecture (and others like it) precisely, and start to use it in cryptography.

4 Computational Model

4.1 Algorithms

Informally, we all know what an algorithm is: something that you can implement in your favorite programming language, and run on any kind of computer (or network of computers) you might like. As a formal mathematical object, an algorithm is a *Turing machine*. In this course we will not need the formal definition too often, and the informal notion will usually suffice.

Running time. The *running time* of an algorithm on an input is the number of "basic" steps it performs before terminating. Basic steps include reading or writing to a location in memory, performing an arithmetic operation (on fixed-size pieces of data), etc. For our purposes, the exact set of basic operations will not be too important, because the standard computational models we consider can simulate one another with only polynomial slowdown. That is, an algorithm with running time T in one model can be simulated in another with running time at most dT^c, for some fixed constants $c, d > 0$. The running time $T(\ell)$ of an algorithm is the *maximum* number of steps it performs on any input of length ℓ. We say that an algorithm is *polynomial-time* if $T(\ell) = O(\ell^c)$ for some fixed constant $c \geq 0$.

Randomized algorithms. Our model also allows *randomization*: algorithms may "flip coins." In the Turing machine view, the machine is augmented with a read-only "random tape," which is initialized with an infinite string of uniformly random and independent bits. Alternatively, we can view the algorithm's randomness as an extra argument $r \in \{0,1\}^*$, and we denote the output of $\mathcal{A}$ on input x with "coins" r as $\mathcal{A}(x;r)$. (When omitted, r is taken to be a uniformly random string.) For fixed x, $\mathcal{A}(x)$ defines a distribution over outputs induced by its random coins. If the input is itself random, the output distribution also accounts for that randomness. For a probabilistic algorithm, the running time $T(\ell)$ is the *maximum* number of steps over all inputs of length ℓ and all choices of random coins. An algorithm is *probabilistic polynomial-time* (PPT) if $T(\ell) = O(\ell^c)$ for some fixed constant $c \geq 0$. Note that a PPT algorithm can read only polynomially many bits of its random string.

In this course, PPT algorithms will be our main model of "efficient computation" for cryptographic schemes. This is primarily to give us a *useful, well-behaved abstraction*, which allows us not to get too caught up in the details of implementations. Of course, an algorithm with running time $T(\ell) = \ell^{100}$ is not very useful in practice, and at times we will take a more precise view of running times.

The security parameter. To study how security scales, we index cryptographic schemes and experiments by a positive integer n, called the *security parameter*. Informally, it lets us specify "how much" security we want; it may determine quantities such as the key length, but need not equal the length of an algorithm's input. We give algorithms an additional input 1^n, the all-1s string of length n, so that they can use this parameter. We often suppress this input in notation, but it is implicitly available to all algorithms, including adversaries and reductions. The unary encoding of 1^n allows a PPT algorithm to run for polynomially many steps in n, regardless of the lengths of its other inputs. When the other inputs have length polynomially bounded in n, polynomial time in the total input length is equivalent to polynomial time in n.

Uniformity and advice. All algorithms in these notes, including adversaries, simulators, and reductions, are *uniform* unless explicitly stated otherwise: a single finite program handles every security parameter and input. In particular, the unqualified term "PPT" means uniform probabilistic polynomial time.

To model adversaries that have additional information depending on the security parameter, we can optionally allow *non-uniformity*. A non-uniform algorithm $\mathcal{A}$ has a sequence of advice strings $w_1, w_2, \ldots \in \{0,1\}^*$, and at security parameter n it is also given w_n. The advice may depend on n, but not on the particular input or random coins, and need not be computable from n. For a non-uniform PPT algorithm, the advice length is polynomially bounded in n, and the running time is polynomial in the total input length. Security against non-uniform PPT adversaries is potentially stronger than security against PPT adversaries; we will explicitly identify any use of this stronger requirement.

4.2 Asymptotics

We use asymptotic bounds to describe both computational resources and probabilities as the security parameter grows.

Polynomial and negligible bounds. We say that a function $T(n)$ is *polynomially bounded*—or just *polynomial* for short—written $T(n) = \mathrm{poly}(n)$, if $T(n) = O(n^c)$ for some fixed constant c. A nonnegative function $\nu(n)$ is *negligible*, written $\nu(n) = \mathrm{negl}(n)$, if it vanishes faster than the inverse of any polynomial: $\nu(n) = O(n^{-c})$ for every constant $c > 0$, or equivalently, $\lim_{n\to\infty} \nu(n) \cdot n^c = 0$ for every constant $c > 0$. We usually apply the concept of a negligible function to quantify the probability of an "extremely rare" event, i.e., one that will effectively "never occur" in a polynomial-time universe. As with polynomial time, the concept of a negligible function is a *useful* and *well-behaved* abstraction, though at times we will need to be more precise in our manipulations of probabilities.

Two closure properties will let us combine negligible bounds in security proofs. A sum of a constant number of negligible functions is negligible: for every constant $c > 0$, each summand is $O(n^{-c})$, so their sum is too. The product of a negligible function $\nu(n)$ and a nonnegative polynomially bounded function $p(n)$ is also negligible. Indeed, if $p(n) = O(n^a)$ for some constant $a \geq 0$, then for every constant $c > 0$, we have $\nu(n) = O(n^{-(a+c)})$ and hence $p(n)\nu(n) = O(n^{-c})$.

Question 1. Is the product of a negligible function and a non-negligible function necessarily negligible, necessarily non-negligible, or not necessarily either?

Changing the security parameter. In many settings the effective security parameter can change, and we need to determine whether a negligible function remains negligible in the new parameter. This holds whenever the new parameter is polynomially bounded in the original parameter and tends to infinity. Formally, if $\nu(n) = \mathrm{negl}(n)$ and $n' = n'(n) = \mathrm{poly}(n)$ is a positive integer-valued function tending to infinity as $n \to \infty$, then $\nu(n) = \mathrm{negl}(n')$, meaning that $\nu(n) = O((n')^{-d})$ for every constant $d > 0$ as $n' \to \infty$. Indeed, writing $n' = O(n^c)$ for some constant $c > 0$, for every constant $d > 0$ we have $\nu(n) \in O(n^{-cd}) \subseteq O((n')^{-d})$.[1]

5 One-Way Functions

There are many different notions of computational "hardness" in computer science, e.g.: undecidability (exemplified by the Halting Problem), NP-completeness (circuit satisfiability), time/space hierarchies, and others.

In cryptography, we have a few notions of hardness, the most basic of which is *one-wayness*. A *one-way function* (OWF) is often called the "minimal" cryptographic object, because a scheme satisfying almost any interesting computational security notion will imply the existence of a OWF. (This is obviously an informal statement, but a good rule of thumb. We will see some examples later on.) In other words, we can't have (computational) cryptography without one-way functions.

Definition 5.1. A function $f : \{0,1\}^* \to \{0,1\}^*$ is *one-way* if it satisfies the following conditions.

- *Easy to compute.* There is an efficient algorithm computing f. Formally, there exists a (uniform, deterministic) poly-time algorithm F such that $\mathsf{F}(x) = f(x)$ for all $x \in \{0,1\}^*$.

[1] The bound $n' = \mathrm{poly}(n)$ matters: e.g., if $n' = 2^n$, then 2^{-n} is negligible in n but equals $1/n'$, which is not $O((n')^{-2})$.

- *Hard to invert.* An efficient algorithm inverts f on a random input with only negligible probability. Formally, for any PPT algorithm $\mathcal{I}$, the *advantage* of $\mathcal{I}$

$$\mathbf{Adv}_f(\mathcal{I}) := \Pr_{x \leftarrow \{0,1\}^n}\left[\mathcal{I}(1^n, f(x)) \in f^{-1}(f(x))\right] = \mathrm{negl}(n)$$

  is a negligible function (in the security parameter n).

Some remarks on this definition:

1. On $f^{-1}(f(x))$ versus x: notice that the inverter $\mathcal{I}$ "wins" if it outputs *any* preimage of $f(x)$, i.e., any element in the set $f^{-1}(f(x))$. (The preimage need not even have length n.) This is mainly to rule out trivial function, such as the function f that just maps every input to 0. If we required the inverter to output the *original* $x \in \{0,1\}^n$, then the best any inverter could do against this f would be to guess randomly, which succeeds with only negligible probability 2^{-n}. But this f does not satisfy our intuitive idea of a function that is "hard to invert."

2. The choice of $\{0,1\}^*$ as the domain and range of f is mainly for syntactic convenience, and it may be replaced by any finite sets, one for each value of the security parameter n. There should also be an efficient algorithm for sampling from the domain, so that the entire inversion experiment can be executed efficiently.

3. In contrast to some other notions of hardness you may have seen, this definition is inherently *average-case* rather than *worst-case*. A worst-case definition would say that every $\mathcal{I}$ fails to invert $f(x)$ for *some* (possibly rare) value of x. Our definition is potentially much stronger: it says that $\mathcal{I}$ fails to invert $f(x)$ for *almost all* values of $x \in \{0,1\}^n$ (for large enough n).

4. Notice that in addition to the value $f(x)$ to be inverted, the input to $\mathcal{I}$ includes the string 1^n. This is simply a technicality that always allows the inverter to run in time $\mathrm{poly}(n)$, even if $f(x)$ has length much less than n. Usually we will omit this extra argument, with the implicit understanding that all algorithms are given the security parameter (in unary), and may run in time polynomial in it.

Question 2. Notice that Definition 5.1 allows for $\mathcal{I}$ to have some non-zero (positive) advantage, though it must be negligible. Consider a stronger definition that requires $\mathcal{I}$ to have advantage 0 (i.e., it is impossible to find a preimage in polynomial time). Is this definition plausibly satisfiable? Why or why not?

Question 3. Let f be a one-way function. Is the function $g(x) = f(x)\|0$ necessarily one-way?

5.1 Candidates

We have defined the concept of a one-way function, but does such an object exist? First, it can be shown that if a one-way function exists, then $\mathsf{P} \neq \mathsf{NP}$. Since we have no idea how to show that $\mathsf{P} \neq \mathsf{NP}$, *proving* that a one-way function exists is far beyond our reach. But we have many *candidate* functions that we believe to be one-way. Consider these two examples:

1. Subset-sum: define $f_{\text{ss}} : (\mathbb{Z}_N)^n \times \{0,1\}^n \to (\mathbb{Z}_N)^n \times \mathbb{Z}_N$, where $N = 2^n$, as

$$f_{\text{ss}}(a_1, \ldots, a_n, b_1, \ldots, b_n) = (a_1, \ldots, a_n, S = \sum_{i\,:\,b_i=1} a_i \bmod N).$$

Notice that because the a_is are part of the output, inverting f means finding a subset of $\{1, \ldots, n\}$ (corresponding to those b_is equaling 1) that induces the given sum S modulo N.

2. Multiplication: define $f_{\text{mult}} : \mathbb{N}^2 \to \mathbb{N}$ as[2]

$$f_{\text{mult}}(x, y) = \begin{cases} 1 & \text{if } x = 1 \vee y = 1 \\ x \cdot y & \text{otherwise.} \end{cases}$$

Are these one-way functions, according to our definition? This is something for you to think about until the next lecture.

[2]For security parameter n, we use the domain $[1, 2^n] \times [1, 2^n]$. The cases $x = 1$ and $y = 1$ are special to rule out the trivial preimages $(1, xy)$ and $(xy, 1)$ for the output $xy \neq 1$.

Answers

Question 1. Is the product of a negligible function and a non-negligible function necessarily negligible, necessarily non-negligible, or not necessarily either?

Answer. Not necessarily either; it depends on the functions. Consider $f(n) = 2^{-n}$ (which is negligible) and $g(n) = 2^n$ (which is non-negligible). Then, $f(n) \cdot g(n) = 1$, which is clearly non-negligible. On the other hand, consider $h(n) = 1$ (which is non-negligible). Then $f(n) \cdot h(n) = 2^{-n}$, which is negligible.

Question 2. Notice that Definition 5.1 allows for $\mathcal{I}$ to have some non-zero (positive) advantage, though it must be negligible. Consider a stronger definition that requires $\mathcal{I}$ to have advantage 0 (i.e., it is impossible to find a preimage in polynomial time). Is this definition plausibly satisfiable? Why or why not?

Answer. It is impossible to satisfy this modified definition. Consider an $\mathcal{I}$ that ignores the $f(x)$ it is given and simply outputs a uniformly random string from $\{0,1\}^n$ as its "guess" for a preimage of $f(x)$. This strategy is obviously polynomial time, and it has a nonzero success probability of (at least) 2^{-n}. (There is a 2^{-n} probability of guessing the very same value of x as was used to compute $f(x)$, and there might be other n-bit preimages of $f(x)$ as well, which can only increase the probability of success.) Hence, no function can satisfy the stronger definition. In cryptography, when we can't achieve the "ideal" of making a task completely impossible, we usually do the next-best thing by using a negligible function to mean "effectively zero."

Question 3. Let f be a one-way function. Is the function $g(x) = f(x)\|0$ necessarily one-way?

Answer. Yes, it is. Observe that any g-preimage of $g(x) = f(x)\|0$ is also an f-preimage of $f(x)$, so any potential inverter for g is "effectively" also an inverter for f. Since the latter can have only negligible advantage (because f is one-way), the same goes for the former, and hence g is one-way. In future lectures we will formalize this argument using a *reduction*: for any hypothetical efficient g-inverter $\mathcal{I}_g$, we will use it as a "black box" to construct an efficient f-inverter $\mathcal{I}_f$ such that $\mathbf{Adv}_f(\mathcal{I}_f) = \mathbf{Adv}_g(\mathcal{I}_g)$.